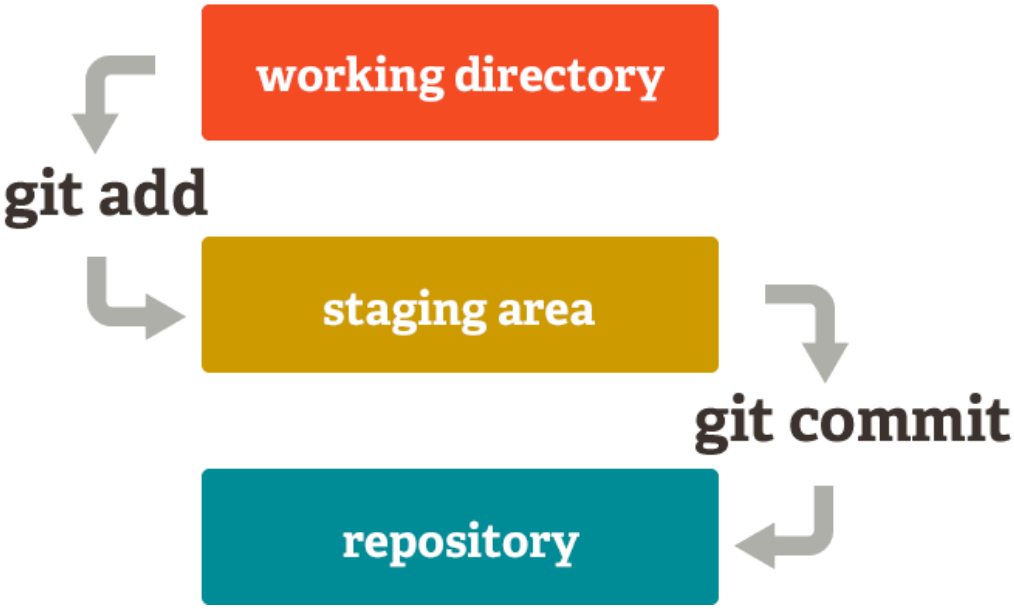
Experiment 1
Aim - To perform git operations
LO-

Theory -



1. The Working Directory (The Sandbox)

This is the folder on your computer (like `C:\Users\15L\Desktop\Devops_Lab1`). It holds the actual files you are currently editing in your IDE.

- **State:** Files here are **Modified** (if you changed them) or **Untracked** (if they are brand new).
- **Concept:** Git knows these files exist or have been changed, but it has *not* securely saved these changes yet. You can break things here freely without affecting your project's history.

2. The Staging Area (The Loading Dock)

Also known as the "Index." This is an invisible middle-ground where you prepare your next snapshot.

- **State:** Files here are **Staged**.

- **Concept:** Imagine you are loading boxes onto a delivery truck. You might only want to ship *some* boxes today, not the whole warehouse. `git add` moves your modified files onto the loading dock. You can carefully review exactly what is on the dock before the truck leaves.

3. The Local Repository (The Vault)

This is the hidden `.git` folder inside your project. It is the actual database where Git permanently stores your project's history and snapshots.

- **State:** Files here are **Committed**.
- **Concept:** When you run `git commit`, the truck leaves the loading dock and gets locked in the vault. A permanent snapshot of your staged files is created, given a unique ID (a SHA-1 hash), and added to your timeline.

4. The Remote Repository (The Cloud)

Everything in the three steps above happens **entirely on your local computer**. You do not need the internet to branch, add, or commit.

The Remote Repository (like GitHub, GitLab, or Bitbucket) is simply a synchronized backup of your Local Repository.

- `git push` copies the snapshots from your local vault up to the remote vault.
- `git pull` downloads new snapshots from the remote vault into your local vault, and immediately updates your Working Directory to match.

1) Git clone - The `git clone` command downloads an exact, linked copy of an existing remote repository from the internet onto your local computer.

```
C:\Users\15L\Desktop\Duplicate_folder>git init
Initialized empty Git repository in C:/Users/15L/Desktop/Duplicate_folder/.git/

C:\Users\15L\Desktop\Duplicate_folder>git clone https://github.com/Viraj7-hub/devops_lab1.git
Cloning into 'devops_lab1'...
remote: Enumerating objects: 10, done.
remote: Counting objects: 100% (10/10), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 10 (delta 0), reused 10 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (10/10), done.

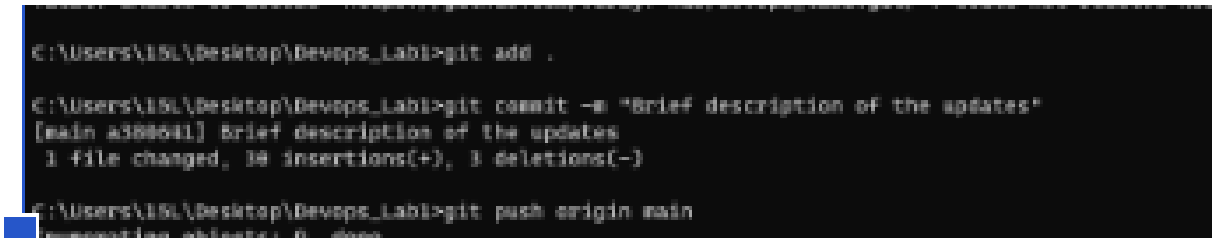
C:\Users\15L\Desktop\Duplicate_folder>
```

2) git init - used to initialise a local repository to git

```
C:\Users\15L\Desktop\Duplicate_folder>git init
Initialized empty Git repository in C:/Users/15L/Desktop/Duplicate_folder/.git/
```

3) git commit - The `git commit` command saves a permanent snapshot of your staged files into your local repository's history, along with a message describing what you changed.

The `git commit` command saves a permanent snapshot of your staged files into your local repository's history, along with a message describing what you changed.



4) git push - The `git push` command uploads your locally saved commits to a remote repository to update it with your latest changes.

```
C:\Users\15L\Desktop\Devops_Lab1>git add .

C:\Users\15L\Desktop\Devops_Lab1>git commit -m "Brief description of the updates"
[main a380641] Brief description of the updates
 1 file changed, 30 insertions(+), 3 deletions(-)

C:\Users\15L\Desktop\Devops_Lab1>git push origin main
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 20 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (5/5), 903 bytes | 903.00 KiB/s, done.
Total 5 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/Viraj7-hub/devops_lab1.git
   1a84293..a380641  main -> main
```

5) git pull - The `git pull` command downloads the latest updates from a remote repository and immediately merges them into your current local working branch.

```
C:\Users\15L\Desktop\Devops_Lab1>git pull origin main
remote: Enumerating objects: 9, done.
remote: Counting objects: 100% (9/9), done.
remote: Compressing objects: 100% (1/1), done.
remote: Total 5 (delta 1), reused 5 (delta 1), pack-reused 0 (from 0)
Unpacking objects: 100% (5/5), 405 bytes | 81.00 KiB/s, done.
From https://github.com/Viraj7-hub/devops_lab1
 * branch            main       -> FETCH_HEAD
   a380641..077123c  main       -> origin/main
Updating a380641..077123c
Fast-forward
 Desktop/Devops_Lab1/Untitled-1.py | 5 ++++-
 1 file changed, 4 insertions(+), 1 deletion(-)

C:\Users\15L\Desktop\Devops_Lab1>
```

`git reset` undoes local changes by moving the current branch pointer (HEAD) back to a specified previous commit, optionally updating the staging area and working tree depending on the flags used.

```
❯ git revert HEAD
Auto-merging...
[main b5c6d7e] Revert "Create a new login page"
 1 file changed, 1 insertion(+), 1 deletion(-)
 create mode 100644 auth/login.html
...
Please enter the commit message for your changes. Lines starting
with '#' will be ignored, and an empty message aborts the commit.
On branch main
Changes to be committed:
```

Git granch - The `git branch` command is used to list, create, rename, or delete branches within a Git repository, allowing you to manage parallel tracks of development.

```
C:\Users\15L\Desktop\Devops_Lab1

C:\Users\15L\Desktop\Devops_Lab1>git branch feature/login

C:\Users\15L\Desktop\Devops_Lab1>git branch
* main
  feature/login

C:\Users\15L\Desktop\Devops_Lab1>git checkout feature/login
Switched to branch 'feature/login'

C:\Users\15L\Desktop\Devops_Lab1>
```

Git merge - The `git merge` command is used to integrate changes from an independent branch into your currently active branch, effectively combining separate lines of development.

